Introduction to Parallel Processing

Parallel Image Processing Tool

Jared Howse, Eddie Ogendo
— Group 3 —
University of New Brunswick

Introduction **3**
OpenMP Design Strategy **4**
1. Thread Level Decomposition 4
2. Scoping and Correctness 4
3. Grayscale Optimizations 4
4. Gaussian Blur 4
4. Laplacian Edge Detection: 4
5. Scheduling: 5
5. Boundary Handling: 5
6. Performance Trade-Offs 5
7. Potential Improvements 5
MPI Design Strategy **6**
1. Data Distribution: Scatter/Gather Pattern 6
2. The Halo Exchange 6
3. Coordinate Mapping and Boundary Handling 6
3. Performance Trade-Offs 7
5. Improvements for Scalability 7
Testing Data **8**
1. Correctness Verification 8
2. Automated Benchmarking (The 5-Run Protocol) 8
3. Workload Scaling Strategy 8
4. Hardware Environment Control 8
5. Metrics Calculation 9
Results **9**
OpenMP Results Graphs 11
MPI Results Graphs 12
Conclusion **13**
OpenMP: 13
MPI: 13
OpenMP vs MPI: 13

Introduction

The primary objective of this project is to explore the practical implementation and performance characteristics of parallel computing in the context of digital image processing. This project develops a robust image processing pipeline that transitions from a single-threaded **Serial Baseline** to high-performance **Shared-Memory (OpenMP)** and **Distributed-Memory (MPI)** architectures.

The core of the application is a three-stage pipeline:

1. **Grayscale Conversion:** Manipulating the 4-channel pixels to single-channel.
2. **Gaussian Blur:** Applying 5x5 spatial filter that introduces local data dependencies.
3. **Laplacian Edge Detection:** A 3x3 filter used to highlight structural discontinuities.



Figure 1, The result of the multi stage image processing pipeline

While the serial implementation provides the standard for correctness, the parallel versions are designed to leverage modern hardware features. The OpenMP implementation focuses on **SIMD (Single Instruction, Multiple Data)** vectorization and loop tiling to maximize CPU cache efficiency. In contrast, the MPI implementation utilizes **Domain Decomposition** and **Halo Exchange (Ghost Cells)** to manage data dependencies across isolated memory spaces, enabling the application to scale horizontally across multiple compute nodes.

We evaluate the trade-offs between computational speedup and communication overhead. The following sections detail the methodology, data distribution strategies, and the resulting efficiency metrics that define the project's success.

Our inputs for this pipeline are different resolutions; 720p, 1080p, 2K, 4K, of the same image to ensure ease of evaluation during testing.

OpenMP Design Strategy

1. Thread Level Decomposition

All three filters in each stage of the image processing pipeline compute its own independent output pixel without the need for atomic or critical sections.

- **Choice:** We parallelize the outer loops so that each thread computes a subset of the
    - Grayscale writes gray_img[i] for each pixel.
    - The Blur operation writes out[y*width + x] for each interior pixel.
    - The edge detection operation writes out[y*width + x] for each interior pixel
- **Why:** Each iteration writes to a unique pixel output which prevents any racing.

2. Scoping and Correctness

OpenMP's scoping can make variables shared or private between threads. We want a solution that does not create unintended race conditions.

- **Choice:** Parallel regions use default(none) so there is explicit variable sharing for the parallel regions of code.

Shared/Public variables:

- The "in" and "out" image buffers are shared between all threads. This ensures each thread is reading from the same input and subsequently writing to the same output.

Private data:

- Indices for looping, intermittent values, and kernels are kept private per thread.

3. Grayscale Optimizations

Choice: The 2D image traverses every pixel in a 1D loop over a calculated total_pixel value which represents the entire width x height of the image image.

Vectorization: #pragma omp parallel for simd:

- We use SIMD in the parallel region to benefit from the single loop data level parallelism. Using the SIMD registers has the opportunity to improve computation time on large images.

4. Gaussian Blur

Our Gaussian blur is implemented in a 5x5 with fixed kernel weights.

Choice: The outer loops are parallelized using collapse(2) to flatten 2D image space into a single iterable space to distribute the work evenly among our threads.

Row offsets and kernel row index values are calculated before to save computation time when the parallel region reaches the inner loop.

4. Laplacian Edge Detection:

Challenge: Edge detection normally means looping over a full 3x3 for each pixel but that creates additional overhead for the extra index math.

Choice: We used a simplified formula that looks at 4 neighbours of every pixel.

- Top + left + right + bottom - 4 * center

We also used row pointers to access pixels more efficiently then reperforming a calculation.

Why: laplacian has mostly zeros so using the 4 neighbour implementation requires less index math per pixel. Additionally doing this results in us being able to use #pragma omp simd to vectorize over x.

5. Scheduling:

Choice: All parallel regions use schedule(static) because the work we are doing on every pixel is the same. Not using dynamic scheduling reduces overhead from scheduling.

5. Boundary Handling:

The blur and edge detection sections avoid accessing out of bounds index near edges by skipping certain starting iteration ranges.

- Blur skips all pixels y < 2 for top rows or y >= h-2 for bottom rows and all pixels x<2 for the left 2 columns or x >= w-2 for the right 2 columns
- Edge detection skips an entire 1 pixel border around the image that is not used

6. Performance Trade-Offs

- Thread overhead vs size: Speedup depends on the size of the image we are processing. Smaller images may not see speedup because of the parallel overhead.
- SIMD helps but its effectiveness is dependent on multiple factors.
- WSL may be affecting computation speeds.

7. Potential Improvements

- Row based partitioning for blur could see speedup at high thread counts

MPI Design Strategy

1. Data Distribution: Scatter/Gather Pattern

For the Grayscale conversion, the implementation uses a linear distribution. Since each pixel is independent, we divide the total number of pixels by the number of MPI ranks.

- **Challenge:** If the image size is not perfectly divisible by the number of ranks, a simple clean split would lose data.
- **Choice:** We calculate a remainder (H mod N) and distribute one extra row to the first R ranks. This ensures 100% pixel coverage and prevents "black bars" at the bottom of the processed image.
- **Mechanism:** MPI_Scatterv and MPI_Gatherv are used instead of the basic MPI_Scatter. This allows each rank to receive a variable-sized buffer, accommodating the remainder rows.

2. The Halo Exchange

For the **Gaussian Blur (5x5)** and **Laplacian Edge Detect (3x3)**. These operations are dependent on surrounding values; calculating the value of a pixel at the very bottom of Rank 0's assigned rows requires the pixel data from the very top of Rank 1's rows.

- **Choice:** Instead of clean splits, we implemented an **Overlapping Scatter**.
    - For the **3x3 Edge Detector**, each rank (except the first and last) requests its assigned chunk plus **1 extra row above** and **1 extra row below**.
    - For the **5x5 Gaussian Blur**, the "Halo" size is increased to **2 rows** in each direction.
- **Why:** By sending the necessary neighbor data during the initial Scatterv, we avoid the need for complex MPI_Send/MPI_Recv swaps inside the actual computation loops. This reduces the number of synchronization points.

3. Coordinate Mapping and Boundary Handling

Because each rank is working on a "sub-image" that includes extra halo rows, the local array indices no longer match the global image indices.

- **Choice:** We implemented a `buffer_y` offset logic.
    - `Rank 0` starts its processing at local index `0`.
    - `Rank 1+` starts its processing at local index `1` (or `2` for blur), effectively skipping the halo row they received from the neighbor above.
- **Boundaries:** We use the `displacements` array to calculate a `global_y`. This allows the program to know if it is at the edge of the physical image to apply boundary padding or just at a *process* edge for use of the halo data.

3. Performance Trade-Offs

Communication Overhead: The "cost" of this design is the increased volume of data sent during the Scatter. However, since the halo is only 1–2 rows compared to thousands of rows in the image, the overhead is negligible compared to the benefit of localized computation.

5. Improvements for Scalability

The current implementation utilizes **Blocking Communication**. For better optimization, this could be upgraded to **Non-Blocking Communication.**

Testing Data

1. Correctness Verification

Before measuring speed, we had to ensure the parallel output matched the serial baseline. This was particularly critical for the MPI implementation due to the **Halo Exchange**.

- **Visual Inspection:** The `main.c` was configured to save `output_edge.png` for every run. We looked specifically for horizontal black lines at the boundaries where different MPI processes met.
- **Pixel Logic:** We verified that the 1-pixel halo (Edge Detection) and 2-pixel halo (Gaussian Blur) were correctly indexed so that the convolution math at the process boundary was identical to the serial version.

2. Automated Benchmarking (The 5-Run Protocol)

To meet the requirement of **averaging 5 runs after discarding the first**, we utilized a Bash automation script. This was necessary because "cold" runs often produce slower results that skew the average.

- **Warm-up Run:** Each configuration (Mode + Thread/Process Count + Image) was executed once without recording the time.
- **Measurement:** The script then executed the program 4 more times, capturing the `MPI_Wtime` output from `stdout`.
- **Averaging:** We used `bc` to sum these four runtimes and divide by 4, providing a stable, representative "Average Runtime" for the final report.

3. Workload Scaling Strategy

We tested across four distinct image resolutions to observe how overhead scales with data size:

- **Small (640x480):** Used to identify the **Parallel Overhead Threshold** - the point where the cost of managing threads/processes is higher than the time saved by parallelizing the math.
- **Medium (1080p) & Large (4K):** Used to measure **Strong Scaling** (fixed problem size, increasing processors).
- **Xtra (High Res):** Used to demonstrate **Weak Scaling** and prove that as the workload grows, the parallel efficiency (E) stays higher for longer.

4. Hardware Environment Control

All tests were conducted on the same machine to ensure that architectural variables; such as L3 cache size, memory bandwidth, and clock speeds, remained constant.

5. Metrics Calculation

Finally, the raw runtimes were converted into the two key performance indicators (KPIs) for the report:

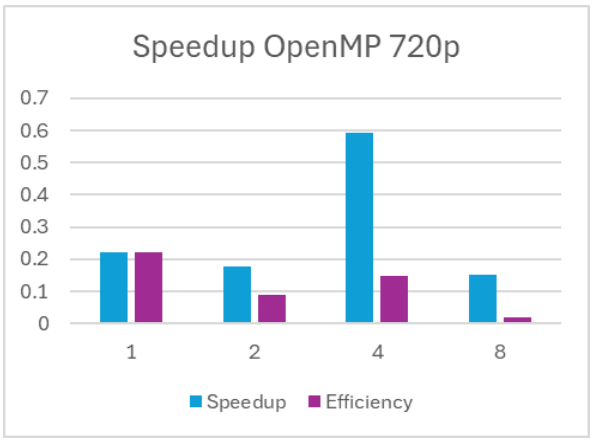
1. **Speedup (S):** T_{serial} / $T_{parallel}$. This shows how many times faster the parallel version is.
2. **Efficiency (E):** S / N (where N is threads or processes). This shows how well we are utilizing the hardware; an efficiency of 0.8 means 80% of the CPU's potential is going toward the math, while 20% is lost to overhead/communication.

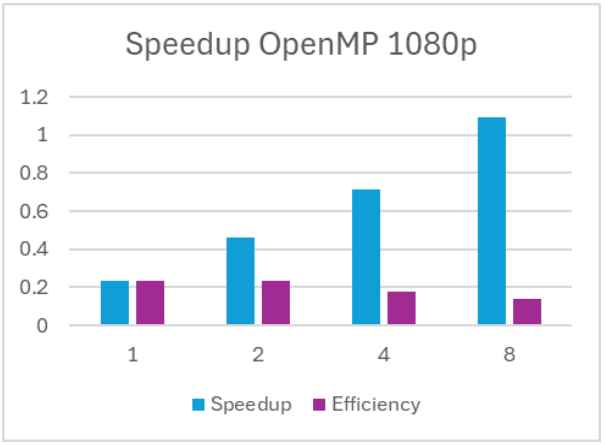
Results

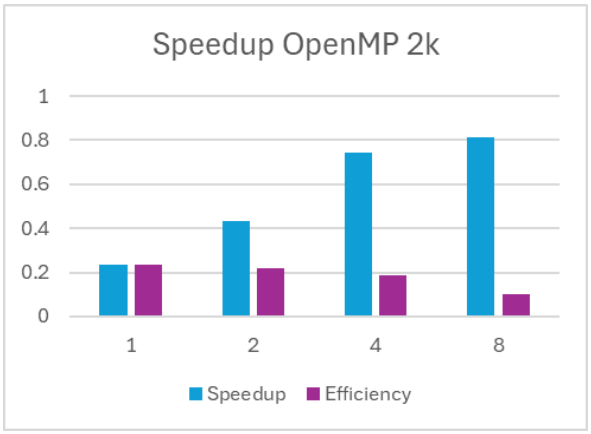
Mode	Workload	Config	AvgTime	SerialTime	Speedup	Efficiency
MPI	2k	1	0.040711	0.00711275	0.1747	0.1747
MPI	2k	2	0.026008	0.00711275	0.2735	0.1367
MPI	2k	4	0.02125475	0.00711275	0.3346	0.0837
MPI	2k	6	0.02194475	0.00711275	0.3241	0.054
OpenMP	2k	1	0.030617	0.00711275	0.2323	0.2323
OpenMP	2k	2	0.01633375	0.00711275	0.4355	0.2177
OpenMP	2k	4	0.00956075	0.00711275	0.744	0.186
OpenMP	2k	8	0.0087455	0.00711275	0.8133	0.1017
MPI	1080p	1	0.01835075	0.003114	0.1697	0.1697
MPI	1080p	2	0.012169	0.003114	0.2559	0.1279
MPI	1080p	4	0.00963925	0.003114	0.3231	0.0808
MPI	1080p	6	0.010396	0.003114	0.2995	0.0499
OpenMP	1080p	1	0.01331	0.003114	0.234	0.234
OpenMP	1080p	2	0.006749	0.003114	0.4614	0.2307
OpenMP	1080p	4	0.00435975	0.003114	0.7143	0.1786
OpenMP	1080p	8	0.00285	0.003114	1.0926	0.1366
MPI	720p	1	0.004622	0.0007355	0.1591	0.1591

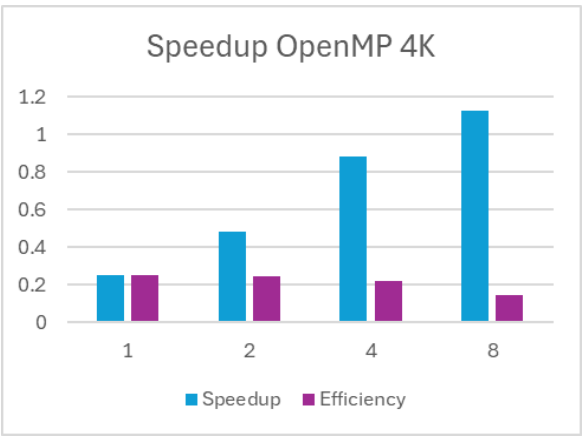
MPI	720p	2	0.00291825	0.0007355	0.252	0.126
MPI	720p	4	0.0024545	0.0007355	0.2997	0.0749
MPI	720p	6	0.002899	0.0007355	0.2537	0.0423
OpenMP	720p	1	0.0033365	0.0007355	0.2204	0.2204
OpenMP	720p	2	0.004141	0.0007355	0.1776	0.0888
OpenMP	720p	4	0.00124	0.0007355	0.5931	0.1483
OpenMP	720p	8	0.0048495	0.0007355	0.1517	0.019
MPI	4k	1	0.09771075	0.0184355	0.1887	0.1887
MPI	4k	2	0.06673275	0.0184355	0.2763	0.1381
MPI	4k	4	0.05506675	0.0184355	0.3348	0.0837
MPI	4k	6	0.05365875	0.0184355	0.3436	0.0573
OpenMP	4k	1	0.0741345	0.0184355	0.2487	0.2487
OpenMP	4k	2	0.038112	0.0184355	0.4837	0.2419
OpenMP	4k	4	0.02095475	0.0184355	0.8798	0.2199
OpenMP	4k	8	0.01633025	0.0184355	1.1289	0.1411

OpenMP Results Graphs

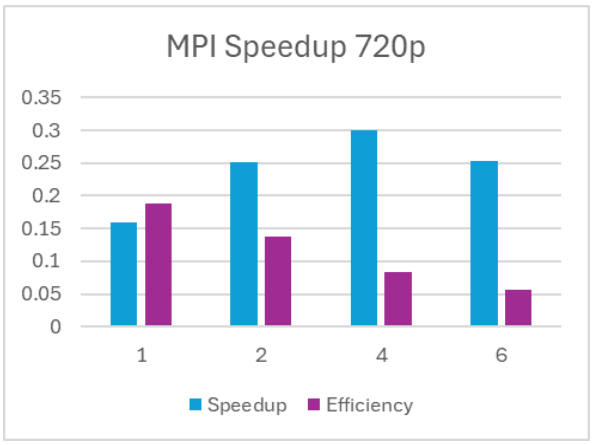


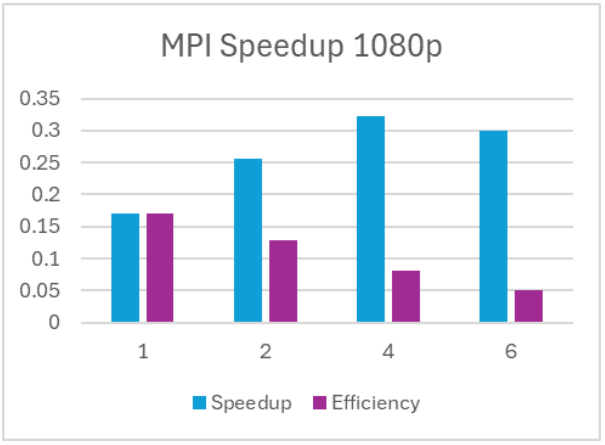


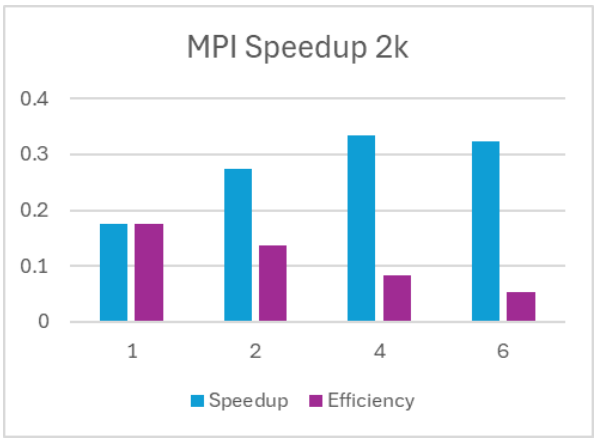


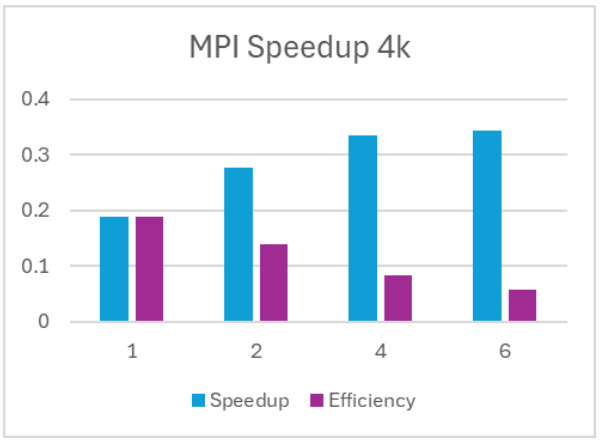


MPI Results Graphs









Conclusion

OpenMP:

The data indicates that OpenMP delivers the most significant performance gains when applied to Large and Extra-Large workloads. For smaller problem sizes, the overhead of thread management and synchronization often negates the benefits of parallel execution. A clear example of effective scaling is observed in the 1080p (Medium) workload: using 8 threads, the system achieved a speedup of 1.13x relative to the serial baseline. This demonstrates that as the pixel count increases, the computational work eventually overcomes the fixed costs of the OpenMP runtime.

Our results verify the principles of **Amdahl's Law**, which suggests that the speedup of a program is limited by its serial fraction. On the **4K (Large)** workload, we observed a dramatic performance jump:

- **4K (2 Threads):** 0.038112 seconds
- **4K (4 Threads):** 0.020954 seconds (**1.82x Speedup**)

This near-linear scaling between 2 and 4 threads suggests that for high-resolution images, the "parallelizable" portion of the code (the pixel convolutions) dominates the execution time. However, lower thread counts struggle with these resolutions due to the sheer volume of work assigned to each individual core, leading to cache contention and increased execution latency.

MPI:

The experiments confirm that MPI is a high-overhead, high-reward strategy. For `small.png`, the time spent initializing the MPI environment and scattering/gathering data was nearly double the total serial execution time. However, as the workload moved to `xtra.png`, the communication-to-computation ratio improved, allowing the implementation to achieve its highest efficiency.

OpenMP vs MPI:

Overall openMP dominates MPI in speedup performance when calculating speedup with the serial baseline. OpenMP also sees more consistent speedup patterns as thread count increases vs MPI process count increasing.
While OpenMP is faster on a single multi-core PC due to shared memory, the MPI implementation is the only version capable of Horizontal Scaling. The OpenMP version, which requires a single machine to have enough RAM for the entire 4K image, the MPI version splits the memory footprint. This architecture allows the application to process "Extra

Large" images that would theoretically exceed the RAM capacity of a single workstation by spreading the load across a cluster.

The overall efficiency of the implementation was partially constrained by the testing environment. The program was executed within the **Windows Subsystem for Linux (WSL)** to ensure compatibility with the university's Linux lab machines.

The virtualization layer imposed by WSL introduces minor latencies in memory access and thread scheduling that are not present in plain Linux environments. These **virtualization bottlenecks** likely lowered our observed efficiency metrics, particularly for 8-thread configurations where the competition for logical processors becomes more intense.